


Proyecto 1 - Entrega 1 – Grupo DevOps X:

Jorge Camilo Chantryt Fernandez

Santiago Castro Duque

Mauricio Arias López

DevOps: Agilizando el Despliegue Continuo de Aplicaciones

2026

# Contenido

Arquitectura ..... 3
Implementación ..... 3
Estructura del proyecto ..... 6
Configuración ..... 7
Despliegues ..... 16
Pruebas ..... 25
Enlaces ..... 30

# Arquitectura

El microservicio *Blacklist Microservice* sigue una arquitectura basada en servicios REST, que permite a diferentes aplicaciones registrar y consultar correos electrónicos mediante solicitudes HTTP. La solución está organizada en tres componentes principales:

- **Capa de presentación y API:** desarrollada con Flask y Flask-RESTful. Expone los *endpoints* encargados de recibir las solicitudes, validar los datos y devolver respuestas en formato JSON.
- **Capa de lógica de negocio:** procesa las solicitudes, normaliza las direcciones de correo electrónico, valida los datos recibidos y controla el registro y la consulta de información. Además, utiliza JWT para proteger el acceso a los *endpoints*.
- **Capa de persistencia:** utiliza PostgreSQL para almacenar los correos electrónicos registrados, el identificador de la aplicación solicitante, el motivo del bloqueo, la dirección IP y la fecha de creación.

La aplicación, desarrollada en Python, utiliza SQLAlchemy para la comunicación con la base de datos y Flask-JWT-Extended para gestionar la autenticación mediante tokens JWT.

Configuración y secretos

- Local: las variables `DATABASE_URL` y `JWT_SECRET_KEY` se cargan desde el archivo `.env` (no versionado en Git).
- AWS Elastic Beanstalk: no se usan esas variables en claro en el código desplegado. El environment define `DB_SECRET_ARN` y `JWT_SECRET_ARN`; al arrancar, la aplicación lee las credenciales y el secreto JWT desde AWS Secrets Manager (vía boto3 y el instance profile). Así se separan configuración local y producción sin exponer secretos en el repositorio.

# Implementación

El microservicio se desarrolla en Python utilizando Flask para la creación de la API REST y PostgreSQL como sistema de gestión de bases de datos. Su implementación contempla dos operaciones principales:

**Registro de correos electrónicos**

- Método: POST

- Endpoint: /blacklists
- Requiere autenticación mediante JWT.
- Recibe el correo electrónico, el UUID de la aplicación y, opcionalmente, el motivo del bloqueo.
- Registra la dirección IP de origen y la fecha de creación.

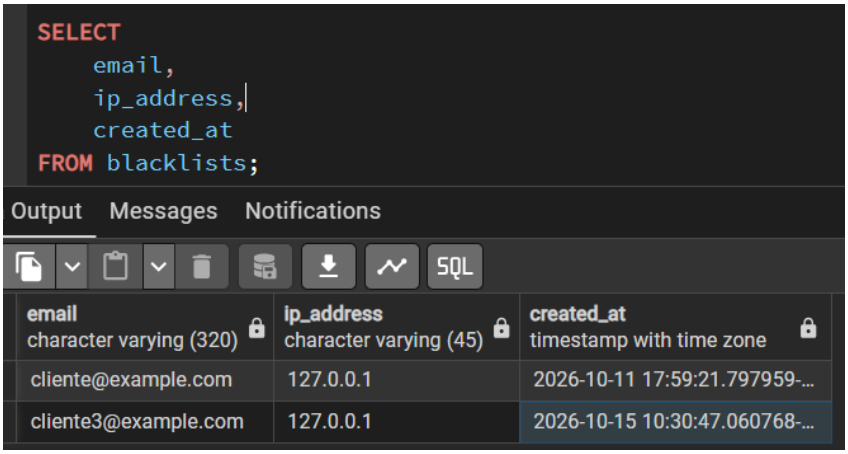



*Ilustración 1 Prueba registro IP y Fecha de creación*

- Almacena la información en PostgreSQL y devuelve una respuesta que confirma el registro.

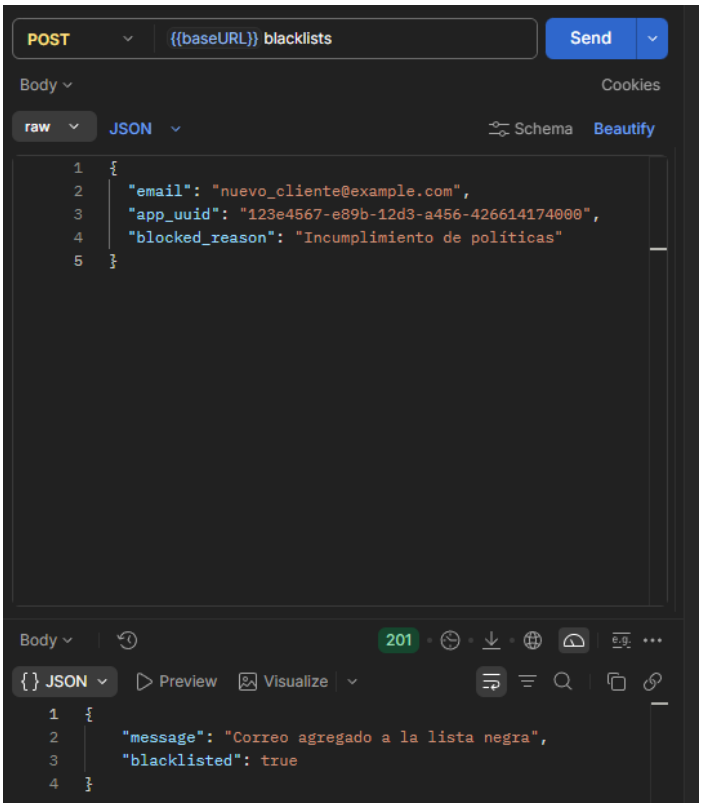



*Ilustración 2 Resultado de operación*

**Consulta de correos electrónicos**

- Método: GET
- Endpoint: /blacklists/<email>
- Requiere autenticación mediante JWT.
- Busca el correo electrónico en la base de datos.
- Devuelve un indicador booleano que señala si el correo está registrado en la lista negra y el motivo asociado, cuando existe.

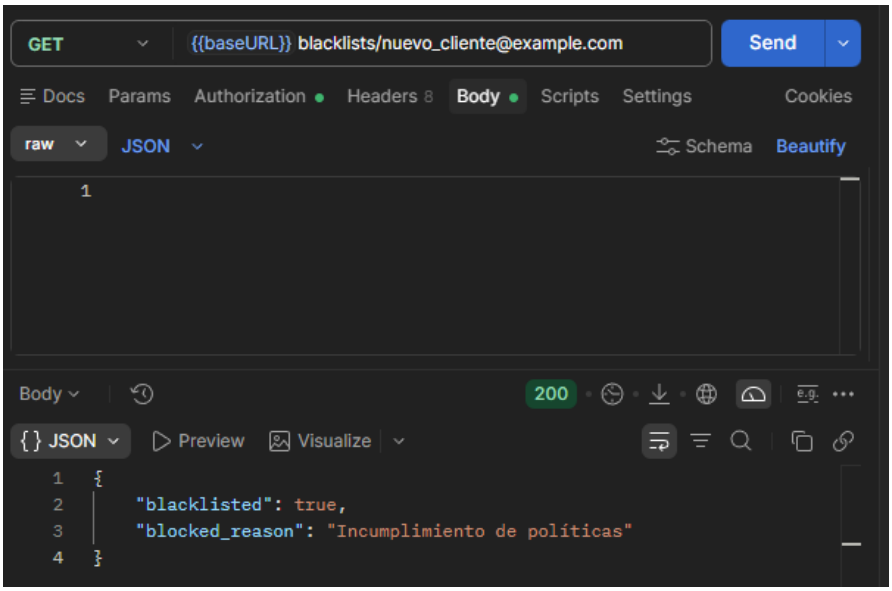



*Ilustración 3 Cumplimiento requerimiento consulta de correos*

Para garantizar la integridad de la información, se valida el formato del correo electrónico, el UUID y la longitud máxima del motivo. Asimismo, se contempla el manejo de registros duplicados para evitar almacenar varias entradas con la misma dirección de correo.

## Estructura del proyecto

El código se organiza en módulos para facilitar su mantenimiento:

- *app/app.py*: Configura Flask, JWT y los endpoints POST/GET /blacklists, / y /health. Expone APP_VERSION e instance para validar estrategias de despliegue.
- *app/config.py*: Local: DATABASE_URL y JWT_SECRET_KEY desde .env. AWS: lee BD y JWT desde Secrets Manager (DB_SECRET_ARN / JWT_SECRET_ARN).
- *application.py*: Punto de entrada que Beanstalk/gunicorn importan como application.
- *app/models/models.py*: define el modelo de datos y la interacción con PostgreSQL mediante *SQLAlchemy*. Persistencia: email, app_uuid, blocked_reason, ip_address, created_at.
- *.ebextensions/options.config*: Inyecta región y ARNs de secretos en Beanstalk.
- *.env* / *.env.example*: contiene las variables de configuración locales. Config local; solo `.env.example` se versiona.
- requirements.txt: especifica las dependencias necesarias para ejecutar el proyecto.

- postman/: está destinada a almacenar las solicitudes y la colección utilizada para probar la API.

# Configuración

## Configuración RDS

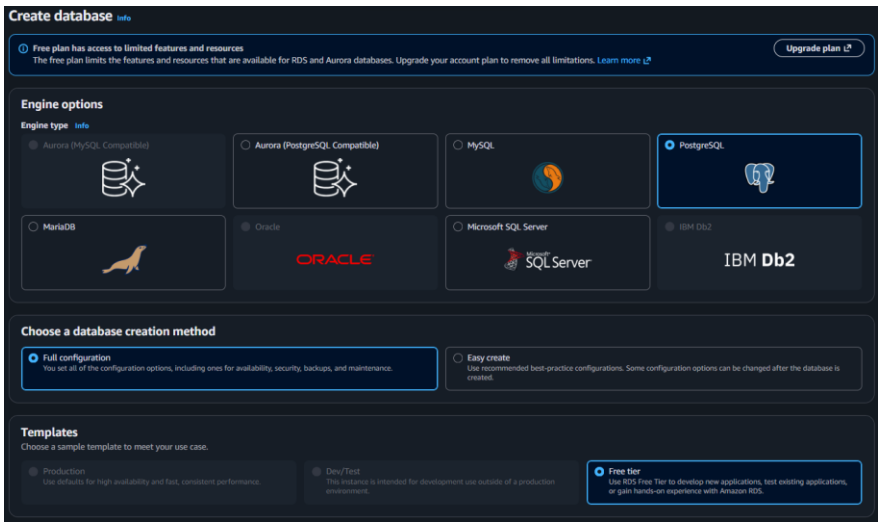



*Ilustración 4 Tipo DB*

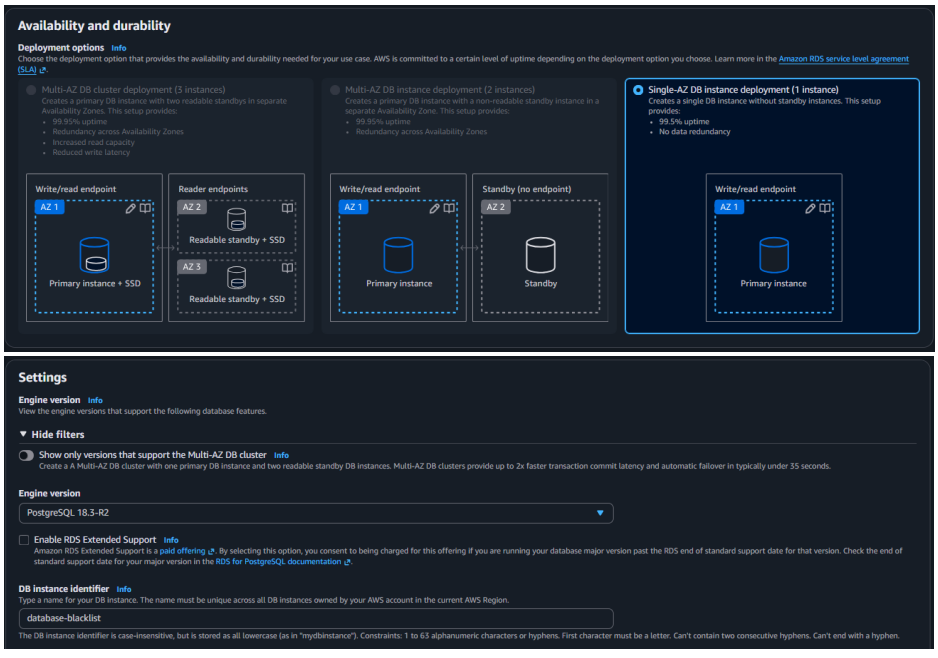



*Ilustración 5 Opciones de despliegue*

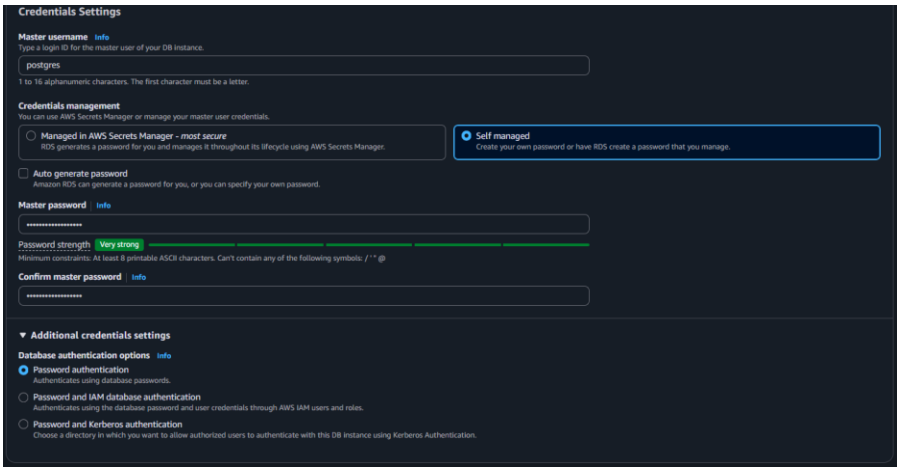



*Ilustración 6 Configuración credenciales*

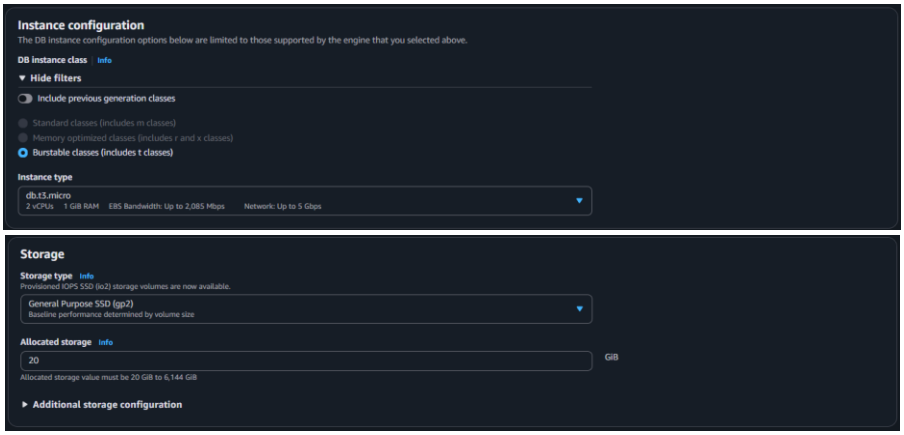



*Ilustración 7 Configuración de almacenamiento*

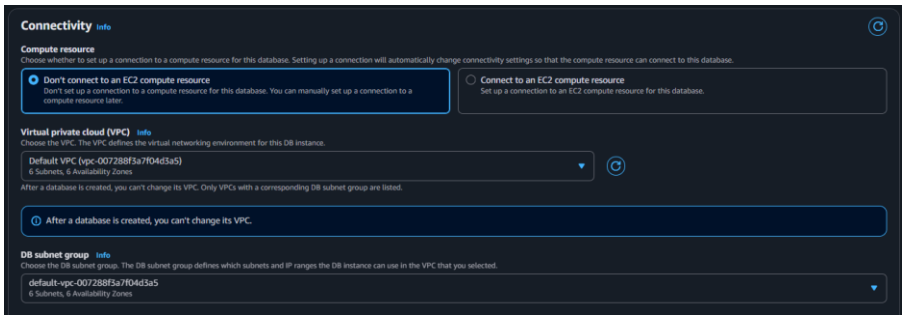



*Ilustración 8 Conexión a la DB*

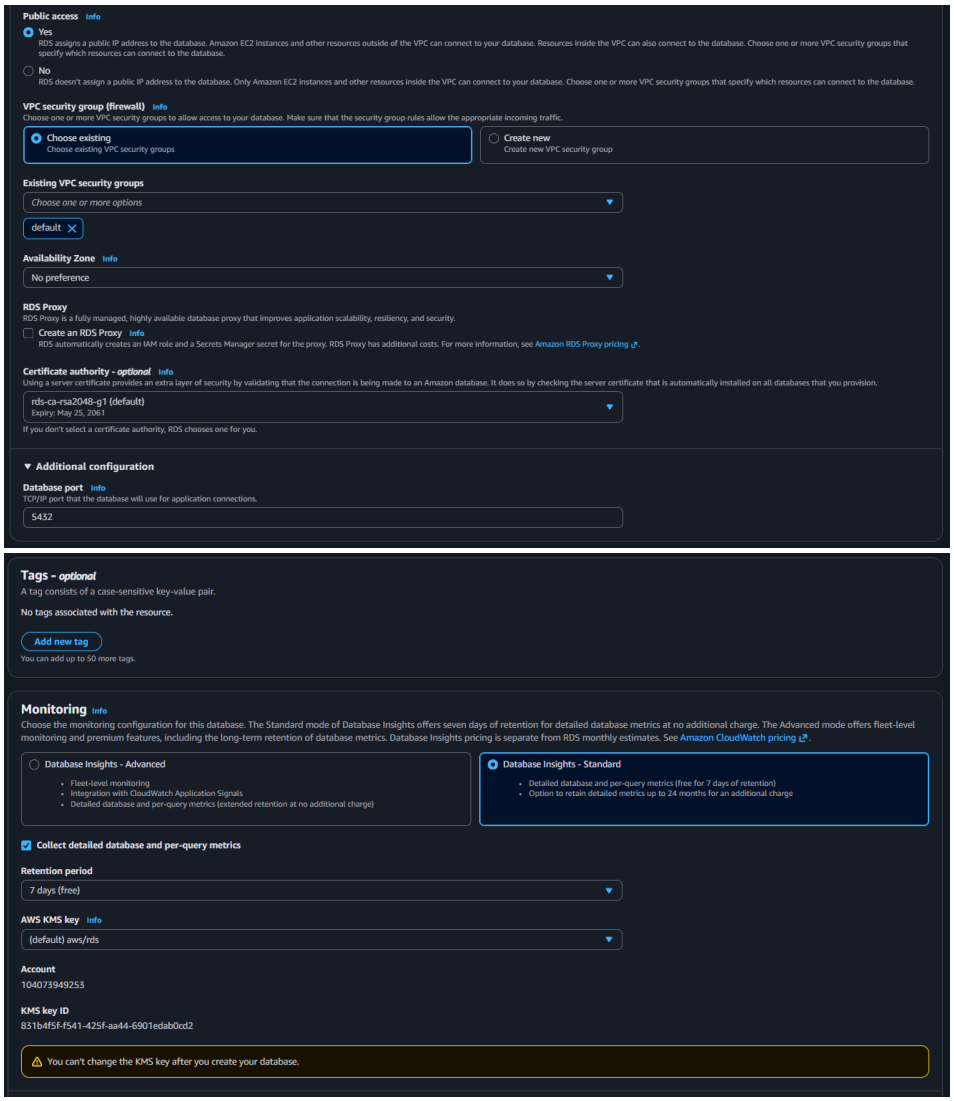






*Ilustración 9 Preferencias*

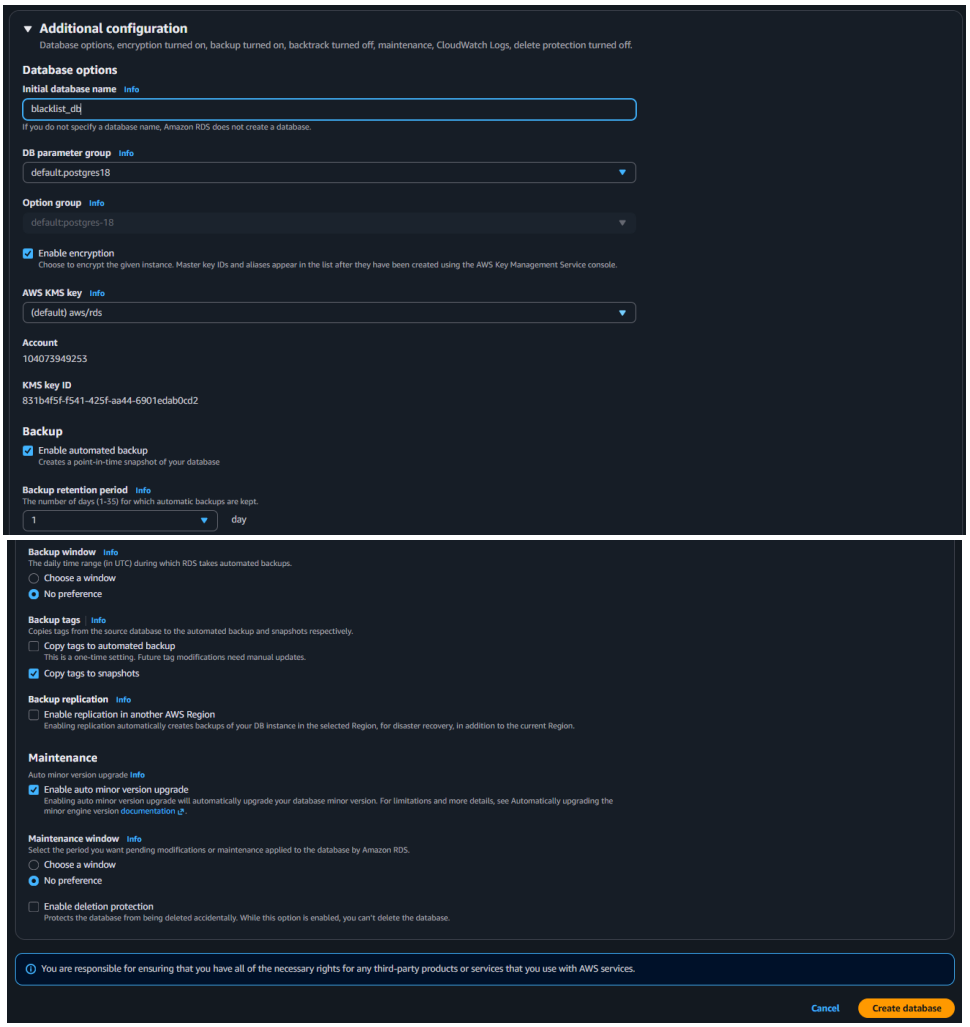



*Ilustración 10 Configuración adicional*

## Configuracion Inbound Rules

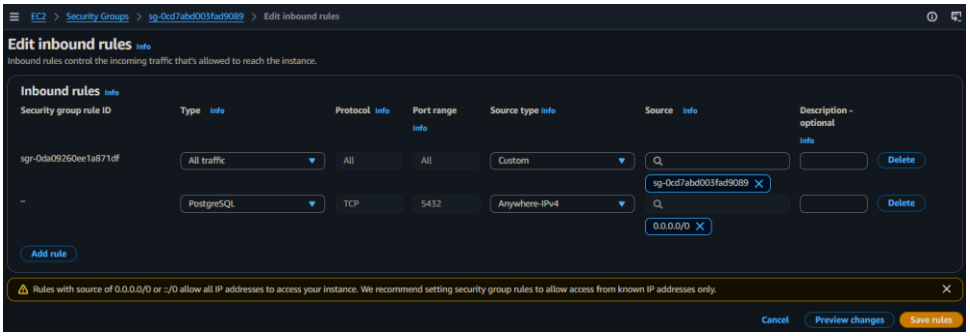



*Ilustración 11 Inbound Rules*

## AWS Secrets Manager

Secretos de la DB:

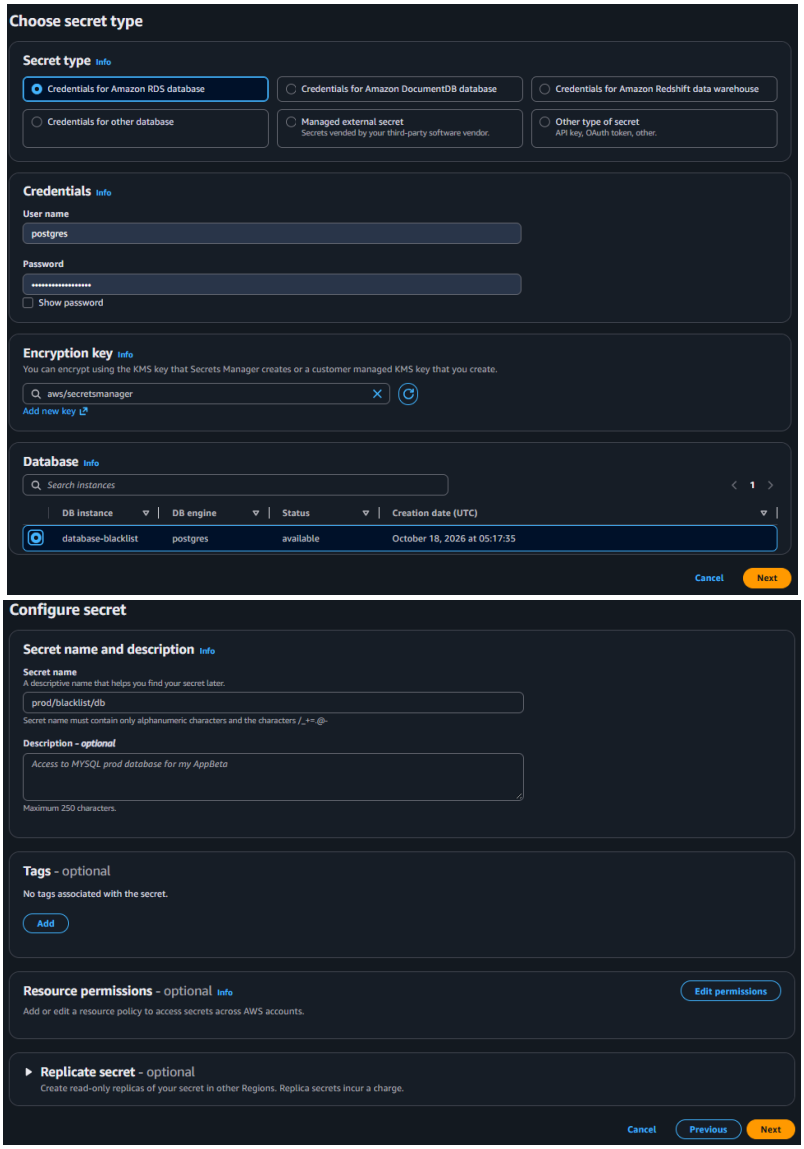



*Ilustración 12 Configuración de secretos de la DB*

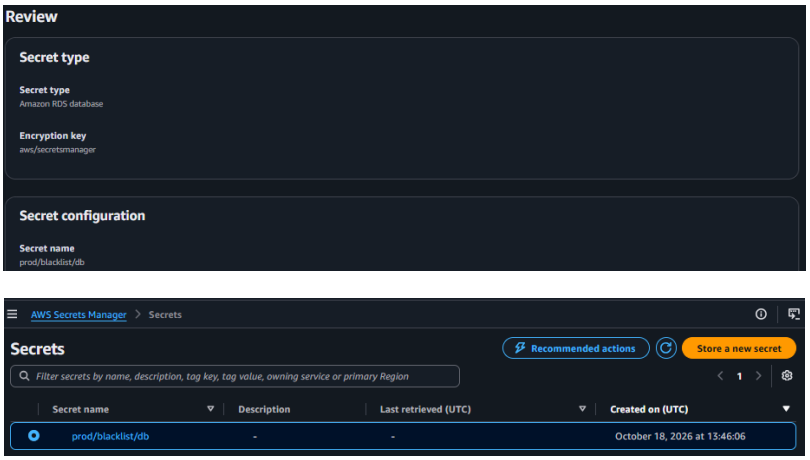



*Ilustración 13 Resumen*

Read Secrets Manager Policy

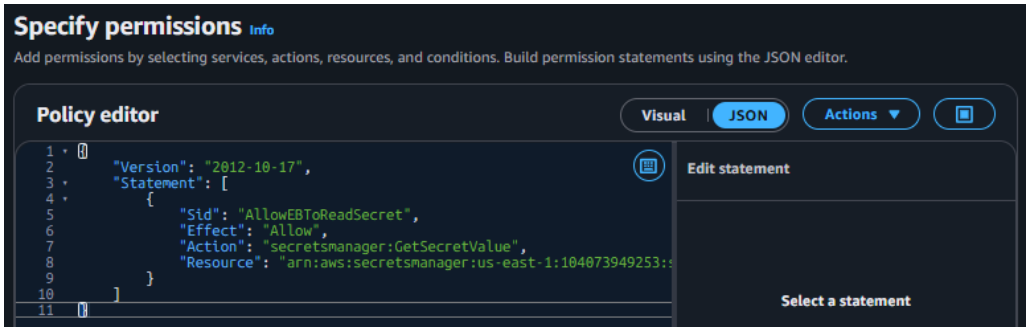
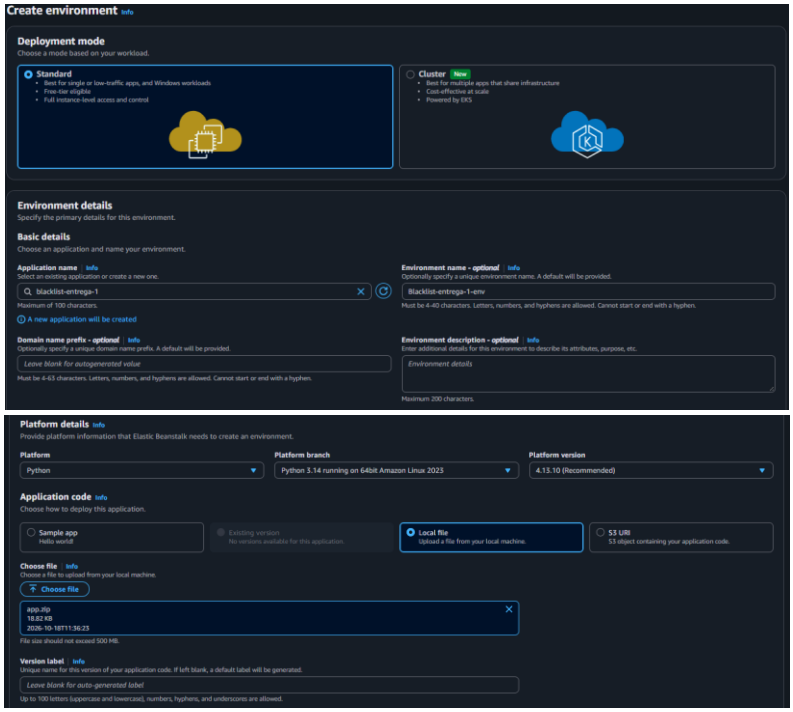


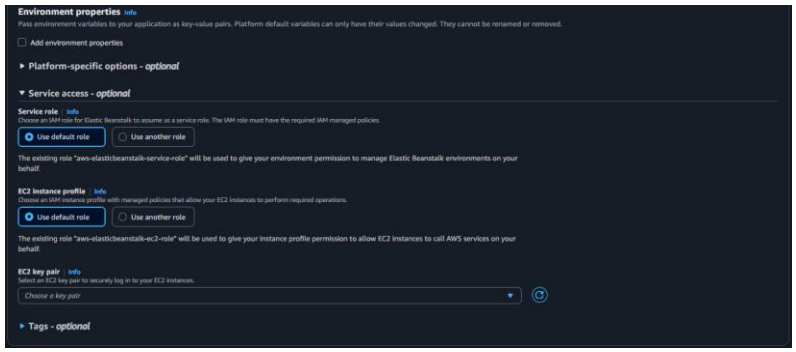

*Ilustración 14 Read Secrets Manager Policy*

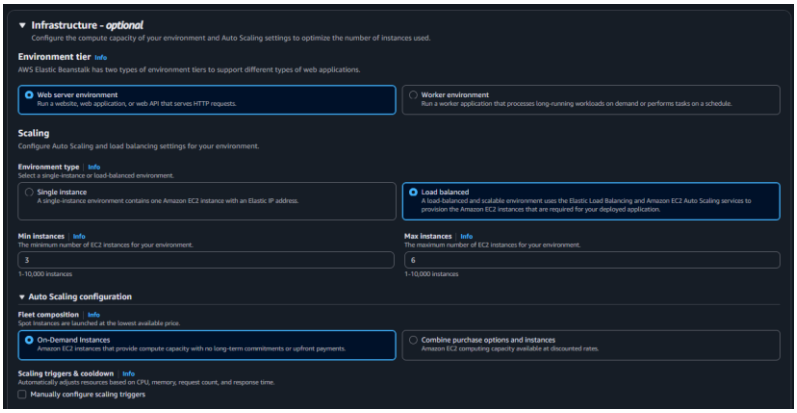
**Configuración del proyecto en AWS Elastic Beanstalk**

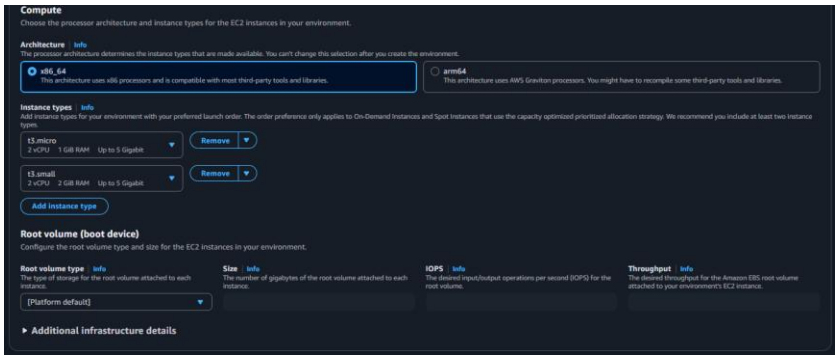
Se creó la aplicación `blacklist-entrega-1` y el environment `Blacklist-entrega-1-env` con las siguientes decisiones:

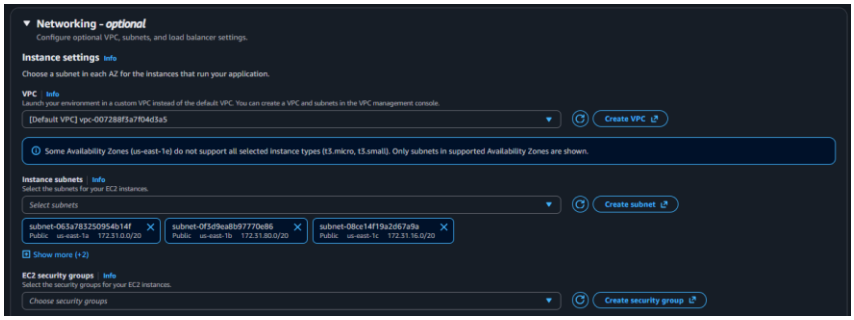
- Plataforma: Python 3.x sobre Amazon Linux 2023.
- Tipo de entorno: Load balanced (Application Load Balancer), no Single instance, para poder ejecutar estrategias con más de una instancia.
- Capacidad (Auto Scaling): mínimo 3 y máximo 6 instancias.
- Paquete de despliegue: ZIP con `application.py`, `Procfile` (gunicorn), `requirements.txt`, `app/` y `.ebextensions/options.config`.
- Variables de entorno del environment: `AWS_REGION`, `AWS_DEFAULT_REGION`, `DB_SECRET_ARN` y `JWT_SECRET_ARN`, apuntando a los secretos en AWS Secrets Manager.
- Permisos: el instance profile del environment tiene política para `secretsmanager:GetSecretValue` sobre los secretos de BD y JWT.
- Base de datos: PostgreSQL en Amazon RDS; las instancias de Beanstalk se conectan vía la URI construida desde el secreto (no se versiona la contraseña en el código).

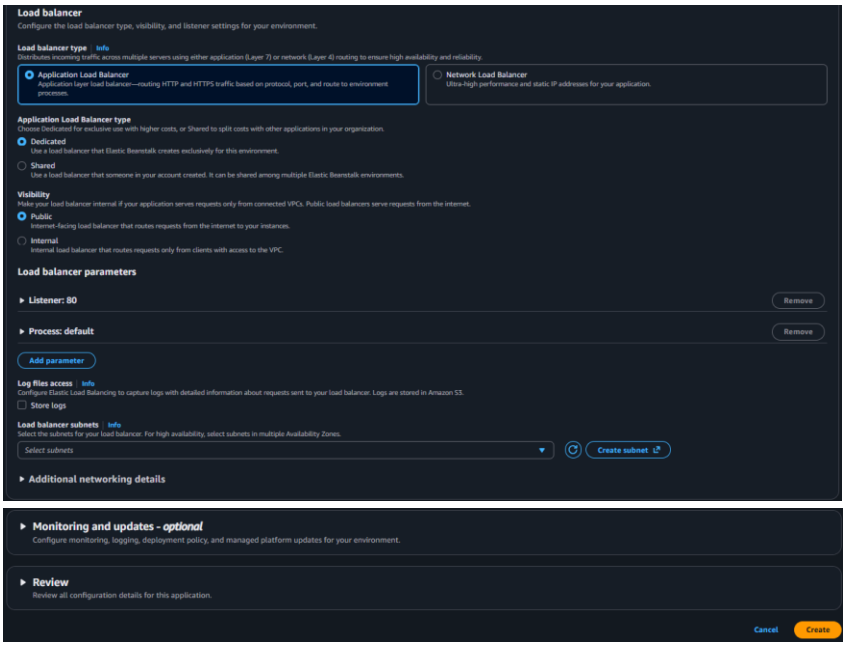



Environment properties Info
Pass environment variables to your application as key-value pairs. Platform default variables can only have their values changed. They cannot be renamed or removed.
Add environment properties
Platform-specific options - optional
Service access - optional
Service role Info
Choose an IAM role for Elastic Beanstalk to assume as a service role. The IAM role must have the required IAM managed policies.
Use default role
Use another role
The existing role "aws-elasticbeanstalk-service-role" will be used to give your environment permission to manage Elastic Beanstalk environments on your behalf.
EC2 instance profile Info
Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.
Use default role
Use another role
The existing role "aws-elasticbeanstalk-ec2-role" will be used to give your instance profile permission to allow EC2 instances to call AWS services on your behalf.
EC2 key pair Info
Select an EC2 key pair to securely log in to your EC2 instances.
Choose a key pair
Tags - optional


Infrastructure - optional
Configure the compute capacity of your environment and Auto Scaling settings to optimize the number of instances used.
Environment tier Info
AWS Elastic Beanstalk has two types of environment tiers to support different types of web applications.
Web server environment
Run a website, web application, or web API that serves HTTP requests.
Worker environment
Run a worker application that processes long-running workloads on demand or performs tasks on a schedule.
Scaling
Configure Auto Scaling and load balancing settings for your environment.
Environment type Info
Select a single-instance or load-balanced environment.
Single instance
A single-instance environment contains one Amazon EC2 instance with an Elastic IP address.
Load balanced
A load-balanced and scalable environment uses the Elastic Load Balancing and Amazon EC2 Auto Scaling services to provision the Amazon EC2 instances that are required for your deployed application.
Min instances Info
The minimum number of EC2 instances for your environment.
3
1-10,000 instances
Max instances Info
The maximum number of EC2 instances for your environment.
6
1-10,000 instances
Auto Scaling configuration
Fleet composition Info
Spot instances are launched at the lowest available price.
On-Demand Instances
Amazon EC2 instances that provide compute capacity with no long-term commitments or upfront payments.
Combine purchase options and instances
Amazon EC2 computing capacity available at discounted rates.
Scaling triggers & cooldown Info
Automatically adjusts resources based on CPU, memory, request count, and response time.
Manually configure scaling triggers


Compute
Choose the processor architecture and instance types for the EC2 instances in your environment.
Architecture Info
The processor architecture determines the instance types that are made available. You can't change this selection after you create the environment.
x86_64
This architecture uses x86 processors and is compatible with most third-party tools and libraries.
arm64
This architecture uses AWS Graviton processors. You might have to recompile some third-party tools and libraries.
Instance types Info
Add instance types for your environment with your preferred launch order. The order preference only applies to On-Demand Instances and Spot Instances that use the capacity optimized prioritized allocation strategy. We recommend you include at least two instance types.
t3.micro
2 vCPU 1 GiB RAM Up to 5 Gigabit
Remove
t3.small
2 vCPU 2 GiB RAM Up to 5 Gigabit
Remove
Add instance type
Root volume (boot device)
Configure the root volume type and size for the EC2 instances in your environment.
Root volume type Info
The type of storage for the root volume attached to each instance.
[Platform default]
Size Info
The number of gigabytes of the root volume attached to each instance.
IOPS Info
The desired input/output operations per second (IOPS) for the root volume.
Throughput Info
The desired throughput for the Amazon EBS root volume attached to your environment's EC2 instance.
Additional infrastructure details

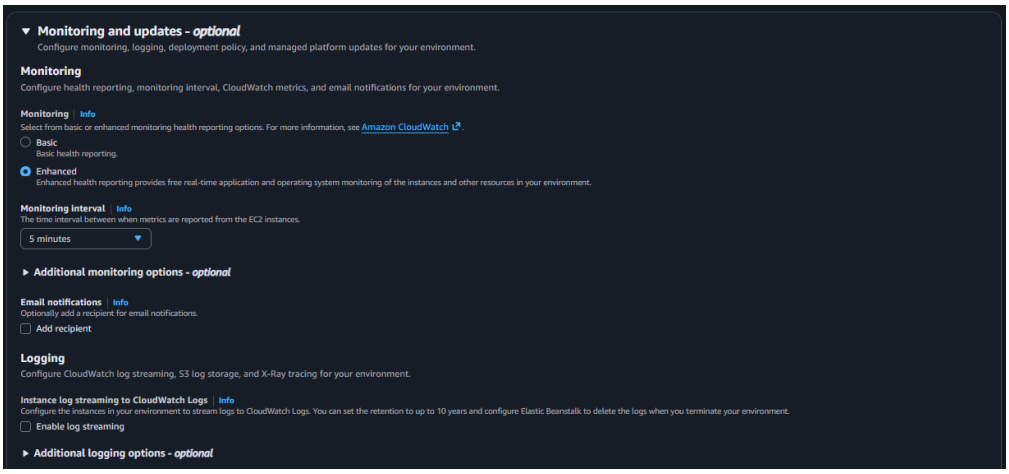

Networking - optional
Configure optional VPC, subnets, and load balancer settings.
Instance settings Info
Choose a subnet in each AZ for the instances that run your application.
VPC Info
Launch your environment in a custom VPC instead of the default VPC. You can create a VPC and subnets in the VPC management console.
[Default VPC] vpc-007288f3a7f04d3a5
Create VPC
Some Availability Zones (us-east-1e) do not support all selected instance types (t3.micro, t3.small). Only subnets in supported Availability Zones are shown.
Instance subnets Info
Select the subnets for your EC2 instances.
Select subnets
Create subnet
subnet-063a783250954b14f
Public us-east-1a 172.31.0.0/20
subnet-0f3d9ea8b97770e86
Public us-east-1b 172.31.80.0/20
subnet-08ce14f19a2d67a9a
Public us-east-1c 172.31.16.0/20
Show more (+2)
EC2 security groups Info
Select the security groups for your EC2 instances.
Choose security groups
Create security group

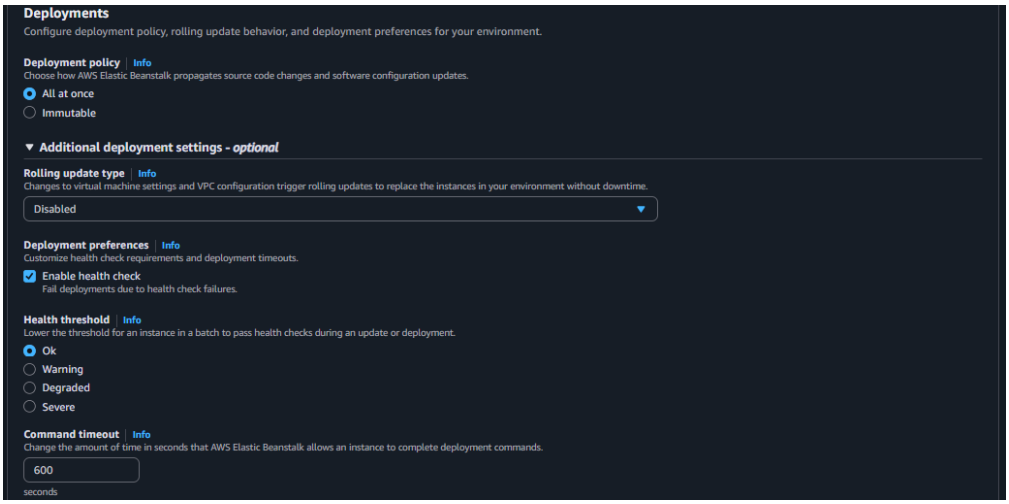



## Configuración health Checks

Se configuró el monitoreo de salud del environment de la siguiente forma:

- Health reporting: Enhanced, para ver el estado por instancia en la consola.
- Ruta de aplicación usada para verificar salud: endpoint `GET /health`, que además de responder HTTP 200 ejecuta un `SELECT 1` contra PostgreSQL. Si la BD no es alcanzable, responde 503.
- El endpoint raíz `GET /` se usa para validar despliegues: retorna `status`, `version` (`APP_VERSION`) e `instance` (hostname), lo que permite comprobar qué instancia y qué versión atiende cada request tras el balanceador.
- Durante los deploys, Beanstalk solo avanza de lote / corta a la flota nueva cuando las instancias pasan los health checks configurados.




# Despliegues

En todos los despliegues se usó un grupo de auto scaling de mínimo 3 y máximo 6 instancias.

Para probar las diferentes estrategias de despliegue se crearon nuevas versiones de la aplicación actualizando solo la version de la app, lo que hicimos fue cambiar la estrategia entre despliegues modificando el Deployment policy desde el Update and Deploy del environment:

## Estrategia all-at-once

Es la estrategia por defecto y fue la primera en ser desplegada

a) Cantidad de instancias utilizadas: mínimo 3, máximo 6.

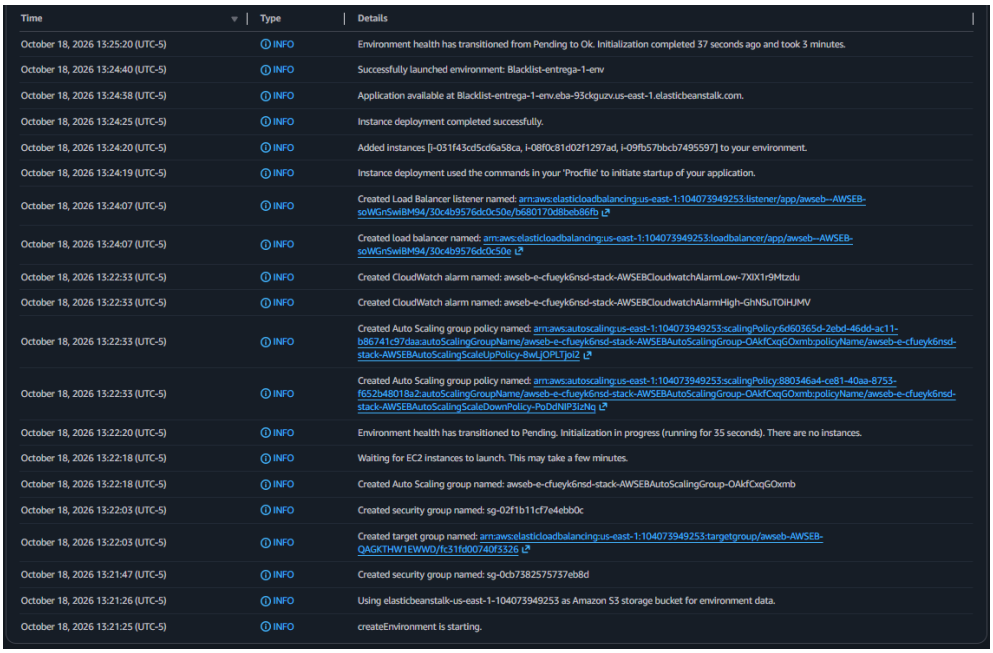
b) Validación: con Postman se hicieron requests al endpoint base y se comprobo la respuesta. Al hacer 3 requests consecutivas se puede apreciar como cambia la dirección de cada instancia, se usan las 3 activas.


```
{
  "instance": "ip-172-31-88-225.ec2.internal",
  "service": "blacklist-microservice",
  "status": "ok",
  "version": "1.0.0"
}
```


c) Tiempo total de despliegue: 3m 15s

d) El despliegue se realizó sobre nuevas instancias.

e) Captura de pantalla del proceso de despliegue:



| Time | Type | Details |
|---|---|---|
| October 18, 2026 13:25:20 (UTC-5) | INFO | Environment health has transitioned from Pending to Ok. Initialization completed 37 seconds ago and took 3 minutes. |
| October 18, 2026 13:24:40 (UTC-5) | INFO | Successfully launched environment: Blacklist-entrega-1-env |
| October 18, 2026 13:24:38 (UTC-5) | INFO | Application available at Blacklist-entrega-1-env.eba-93ckguzv.us-east-1.elasticbeanstalk.com. |
| October 18, 2026 13:24:25 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 13:24:20 (UTC-5) | INFO | Added instances [i-031f43cd5cd6a58ca, i-08f0c81d02f1297ad, i-09fb57bbcb7495597] to your environment. |
| October 18, 2026 13:24:19 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 13:24:07 (UTC-5) | INFO | Created Load Balancer listener named: arn:aws:elasticloadbalancing:us-east-1:104073949253:listener/app/awseb--AWSEB-soWGnSwiBM94/30c4b9576dc0c50e/b680170d8beb86fb |
| October 18, 2026 13:24:07 (UTC-5) | INFO | Created load balancer named: arn:aws:elasticloadbalancing:us-east-1:104073949253:loadbalancer/app/awseb--AWSEB-soWGnSwiBM94/30c4b9576dc0c50e |
| October 18, 2026 13:22:33 (UTC-5) | INFO | Created CloudWatch alarm named: awseb-e-cfueyk6nsd-stack-AWSEBCloudwatchAlarmLow-7XIX1r9Mtzdu |
| October 18, 2026 13:22:33 (UTC-5) | INFO | Created CloudWatch alarm named: awseb-e-cfueyk6nsd-stack-AWSEBCloudwatchAlarmHigh-GhNSuTOiHJMV |
| October 18, 2026 13:22:33 (UTC-5) | INFO | Created Auto Scaling group policy named: arn:aws:autoscaling:us-east-1:104073949253:scalingPolicy:6d60365d-2ebd-46dd-ac11-b86741c97daa:autoScalingGroupName/awseb-e-cfueyk6nsd-stack-AWSEBAutoScalingGroup-OAkfCxqGOxmb:policyName/awseb-e-cfueyk6nsd-stack-AWSEBAutoScalingScaleUpPolicy-8wLjOPLTjoi2 |
| October 18, 2026 13:22:33 (UTC-5) | INFO | Created Auto Scaling group policy named: arn:aws:autoscaling:us-east-1:104073949253:scalingPolicy:880346a4-ce81-40aa-8753-f652b48018a2:autoScalingGroupName/awseb-e-cfueyk6nsd-stack-AWSEBAutoScalingGroup-OAkfCxqGOxmb:policyName/awseb-e-cfueyk6nsd-stack-AWSEBAutoScalingScaleDownPolicy-PoDdNIP3izNq |
| October 18, 2026 13:22:20 (UTC-5) | INFO | Environment health has transitioned to Pending. Initialization in progress (running for 35 seconds). There are no instances. |
| October 18, 2026 13:22:18 (UTC-5) | INFO | Waiting for EC2 instances to launch. This may take a few minutes. |
| October 18, 2026 13:22:18 (UTC-5) | INFO | Created Auto Scaling group named: awseb-e-cfueyk6nsd-stack-AWSEBAutoScalingGroup-OAkfCxqGOxmb |
| October 18, 2026 13:22:03 (UTC-5) | INFO | Created security group named: sg-02f1b11cf7e4ebb0c |
| October 18, 2026 13:22:03 (UTC-5) | INFO | Created target group named: arn:aws:elasticloadbalancing:us-east-1:104073949253:targetgroup/awseb-AWSEB-QAGKTHW1EWWD/fc31fd00740f3326 |
| October 18, 2026 13:21:47 (UTC-5) | INFO | Created security group named: sg-0cb7382575737eb8d |
| October 18, 2026 13:21:26 (UTC-5) | INFO | Using elasticbeanstalk-us-east-1-104073949253 as Amazon S3 storage bucket for environment data. |
| October 18, 2026 13:21:25 (UTC-5) | INFO | createEnvironment is starting. |

*Ilustración 15 Proceso de despliegue all-at-once*

f) Hallazgos encontrados de la estrategia: Es la política por defecto del environment. Actualiza todas las instancias al mismo tiempo sobre la flota existente, por lo que fue la estrategia más rápida. Como no hay lotes, puede haber una breve pérdida de disponibilidad o degradación mientras las instancias reinician el proceso de la aplicación. No crea un Auto Scaling group paralelo (a diferencia de Immutable). Es adecuada si se prioriza velocidad y se tolera un downtime corto.

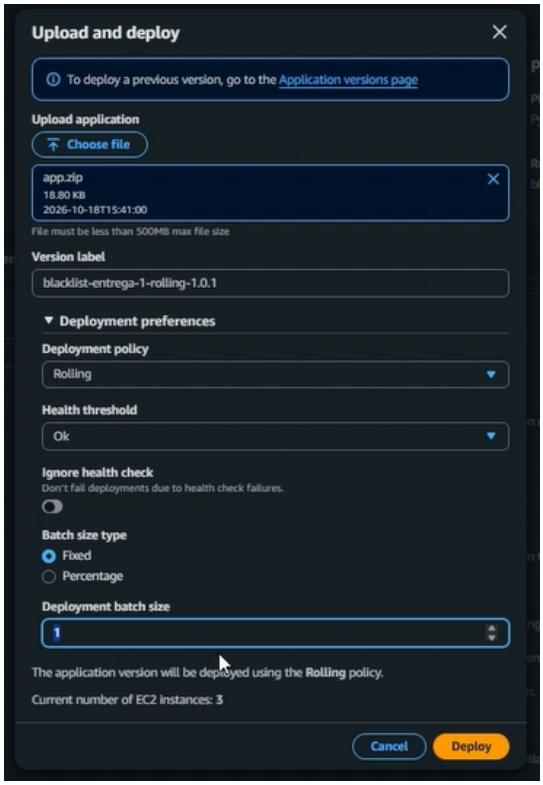
## Estrategia Rolling

Actualiza por lotes (batch). Mientras un lote se actualiza, el resto sigue con la versión vieja. Sin downtime total, pero baja capacidad temporalmente. Usamos batch size 1 entonces va actualizando de a una instancia a la vez, hay momentos en los que saca una instancia para actualizarla, mientras solo quedan dos activas, cuando termina de actualizar la instancia ingresa de nuevo al Load Balancer.

a) Cantidad de instancias utilizadas: mínimo 3, máximo 6.
b) Validación: con Postman se hicieron requests al endpoint base y se comprobo la respuesta. Al hacer 3 requests consecutivas se puede apreciar como cambia la dirección de cada instancia, se usan las 3 activas.


```
{
  "instance": "ip-172-31-88-225.ec2.internal",
  "service": "blacklist-microservice",
  "status": "ok",
  "version": "1.0.1"
}
```


c) Tiempo total de despliegue: 9m 16s
d) El despliegue se realizó sobre las instancias existentes.
e) Captura de pantalla del proceso de despliegue:





| Time | Severity | Message |
|---|---|---|
| October 18, 2026 15:51:04 (UTC-5) | INFO | Environment update completed successfully. |
| October 18, 2026 15:51:04 (UTC-5) | INFO | Successfully deployed new configuration to environment. |
| October 18, 2026 15:51:04 (UTC-5) | INFO | New application version was deployed to running EC2 instances. |
| October 18, 2026 15:50:14 (UTC-5) | INFO | Batch 3: Completed application deployment. |
| October 18, 2026 15:48:10 (UTC-5) | INFO | Batch 3: Registering instance(s) with the load balancer and waiting for them to be healthy. |
| October 18, 2026 15:48:10 (UTC-5) | INFO | Command execution completed on 3 of 3 instances in environment. |
| October 18, 2026 15:48:10 (UTC-5) | INFO | Batch 3: Completed application deployment command execution. |
| October 18, 2026 15:48:06 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 15:47:58 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 15:47:56 (UTC-5) | INFO | Batch 3: Starting application deployment command execution. |
| October 18, 2026 15:47:34 (UTC-5) | INFO | Batch 3: Starting application deployment on instance(s) [i-09fb57bbcb7495597]. |
| October 18, 2026 15:47:34 (UTC-5) | INFO | Batch 2: Completed application deployment. |
| October 18, 2026 15:45:30 (UTC-5) | INFO | Batch 2: Registering instance(s) with the load balancer and waiting for them to be healthy. |
| October 18, 2026 15:45:30 (UTC-5) | INFO | Command execution completed on 2 of 3 instances in environment. |
| October 18, 2026 15:45:30 (UTC-5) | INFO | Batch 2: Completed application deployment command execution. |
| October 18, 2026 15:45:26 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 15:45:18 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 15:45:16 (UTC-5) | INFO | Batch 2: Starting application deployment command execution. |
| October 18, 2026 15:44:54 (UTC-5) | INFO | Batch 2: Starting application deployment on instance(s) [i-031f43cd5cd6a58ca]. |
| October 18, 2026 15:44:54 (UTC-5) | INFO | Batch 1: Completed application deployment. |
| October 18, 2026 15:42:50 (UTC-5) | INFO | Batch 1: Registering instance(s) with the load balancer and waiting for them to be healthy. |
| October 18, 2026 15:42:50 (UTC-5) | INFO | Command execution completed on 1 of 3 instances in environment. |
| October 18, 2026 15:42:50 (UTC-5) | INFO | Batch 1: Completed application deployment command execution. |
| October 18, 2026 15:42:46 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 15:42:38 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 15:42:37 (UTC-5) | INFO | Batch 1: Starting application deployment command execution. |
| October 18, 2026 15:42:14 (UTC-5) | INFO | Batch 1: Starting application deployment on instance(s) [i-08f0c81d02f1297ad]. |
| October 18, 2026 15:42:13 (UTC-5) | INFO | Deploying new version to instance(s). |
| October 18, 2026 15:41:55 (UTC-5) | INFO | Updating environment Blacklist-entrega-1-env's configuration settings. |
| October 18, 2026 15:41:48 (UTC-5) | INFO | Environment update is starting. |

*Ilustración 16 Proceso de despliegue Rolling*

f) Hallazgos encontrados de la estrategia Rolling: es más lenta que all-at-once, hay momentos en los que saca una instancia (batch size=1) para actualizarla, durante ese tiempo solo quedan dos instancias activas, entonces baja la capacidad del servicio temporalmente.

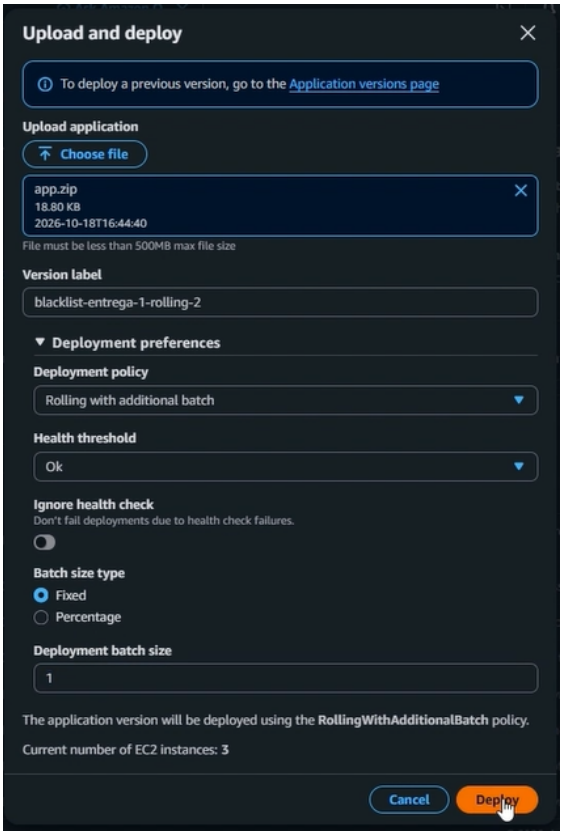
## Estrategia Rolling with Additional Batch

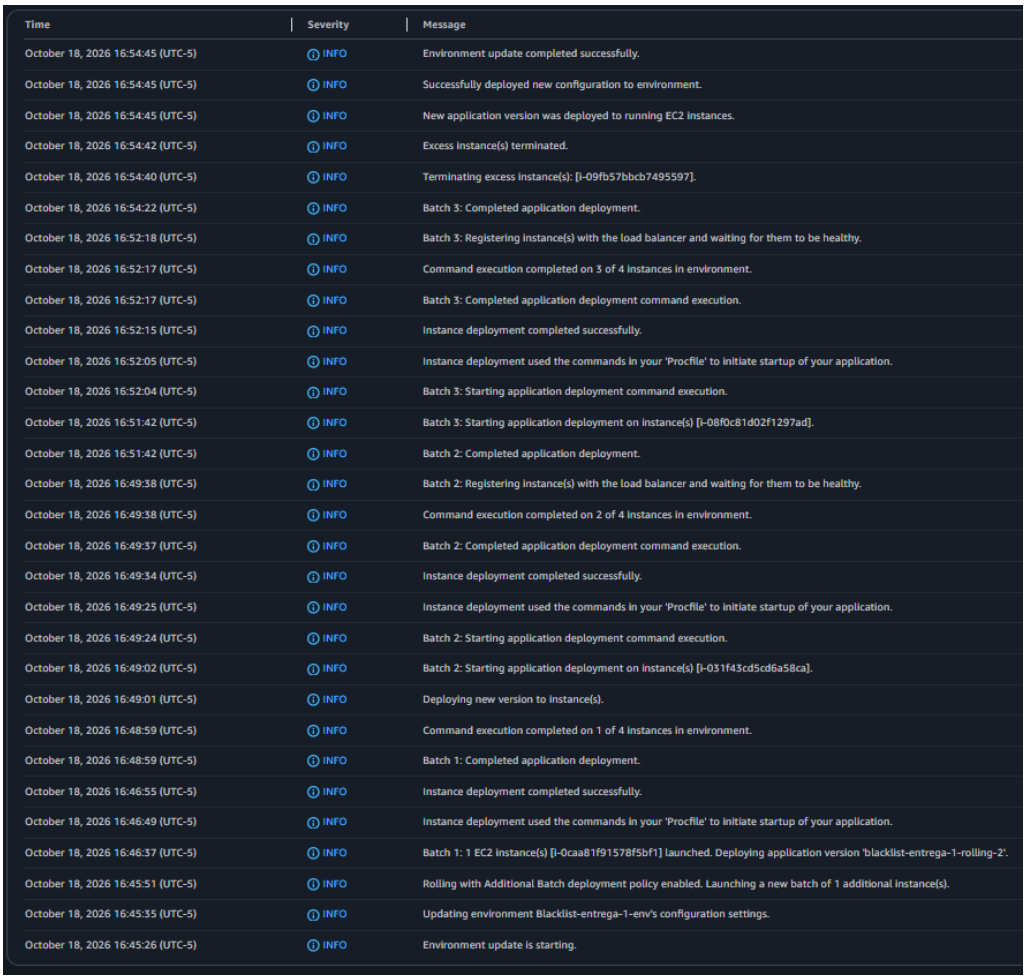
Antes de bajar un lote, lanza un lote extra, hace rolling y al final elimina el extra. Capacidad completa durante el deploy. Más lento. Usa instancias nuevas temporales + las originales.

a) Cantidad de instancias utilizadas: mínimo 3, máximo 6.
b) Validación: con Postman se hicieron requests al endpoint base. Al finalizar, el conteo volvió a 3 instancias y GET / mostró la nueva version (ej. 1.0.3) en todas las respuestas.


```
{
"instance": "ip-172-31-xx-xx.ec2.internal",
"service": "blacklist-microservice",
"status": "ok",
"version": "1.0.3"
}
```


c) Tiempo total de despliegue: 9m 18s
d) El despliegue se realizó usando instancias adicionales nuevas temporales.
e) Captura de pantalla del proceso de despliegue:





| Time | Severity | Message |
|---|---|---|
| October 18, 2026 16:54:45 (UTC-5) | INFO | Environment update completed successfully. |
| October 18, 2026 16:54:45 (UTC-5) | INFO | Successfully deployed new configuration to environment. |
| October 18, 2026 16:54:45 (UTC-5) | INFO | New application version was deployed to running EC2 instances. |
| October 18, 2026 16:54:42 (UTC-5) | INFO | Excess instance(s) terminated. |
| October 18, 2026 16:54:40 (UTC-5) | INFO | Terminating excess instance(s): [i-09fb57bbcb7495597]. |
| October 18, 2026 16:54:22 (UTC-5) | INFO | Batch 3: Completed application deployment. |
| October 18, 2026 16:52:18 (UTC-5) | INFO | Batch 3: Registering instance(s) with the load balancer and waiting for them to be healthy. |
| October 18, 2026 16:52:17 (UTC-5) | INFO | Command execution completed on 3 of 4 instances in environment. |
| October 18, 2026 16:52:17 (UTC-5) | INFO | Batch 3: Completed application deployment command execution. |
| October 18, 2026 16:52:15 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 16:52:05 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 16:52:04 (UTC-5) | INFO | Batch 3: Starting application deployment command execution. |
| October 18, 2026 16:51:42 (UTC-5) | INFO | Batch 3: Starting application deployment on instance(s) [i-08f0c81d02f1297ad]. |
| October 18, 2026 16:51:42 (UTC-5) | INFO | Batch 2: Completed application deployment. |
| October 18, 2026 16:49:38 (UTC-5) | INFO | Batch 2: Registering instance(s) with the load balancer and waiting for them to be healthy. |
| October 18, 2026 16:49:38 (UTC-5) | INFO | Command execution completed on 2 of 4 instances in environment. |
| October 18, 2026 16:49:37 (UTC-5) | INFO | Batch 2: Completed application deployment command execution. |
| October 18, 2026 16:49:34 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 16:49:25 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 16:49:24 (UTC-5) | INFO | Batch 2: Starting application deployment command execution. |
| October 18, 2026 16:49:02 (UTC-5) | INFO | Batch 2: Starting application deployment on instance(s) [i-031f43cd5cd6a58ca]. |
| October 18, 2026 16:49:01 (UTC-5) | INFO | Deploying new version to instance(s). |
| October 18, 2026 16:48:59 (UTC-5) | INFO | Command execution completed on 1 of 4 instances in environment. |
| October 18, 2026 16:48:59 (UTC-5) | INFO | Batch 1: Completed application deployment. |
| October 18, 2026 16:46:55 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 16:46:49 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 16:46:37 (UTC-5) | INFO | Batch 1: 1 EC2 instance(s) [i-0caa81f91578f5bf1] launched. Deploying application version 'blacklist-entrega-1-rolling-2'. |
| October 18, 2026 16:45:51 (UTC-5) | INFO | Rolling with Additional Batch deployment policy enabled. Launching a new batch of 1 additional instance(s). |
| October 18, 2026 16:45:35 (UTC-5) | INFO | Updating environment Blacklist-entrega-1-env's configuration settings. |
| October 18, 2026 16:45:26 (UTC-5) | INFO | Environment update is starting. |

*Ilustración 17 Proceso de despliegue Rolling with Additional Batch*

f) Hallazgos encontrados de la estrategia: Antes de sacar un lote del balanceador, Beanstalk lanza instancias adicionales, hace el rolling y al final termina el lote extra. Por eso se mantiene la capacidad completa (siempre ~3 instancias sirviendo), a costa de un tiempo similar o mayor que Rolling y un costo temporal por las instancias extra. El despliegue combina instancias originales (actualizadas por lotes) con instancias temporales nuevas. Comparado con Rolling (batch=1), no se observó la caída a 2 instancias activas.

## Estrategia Immutable

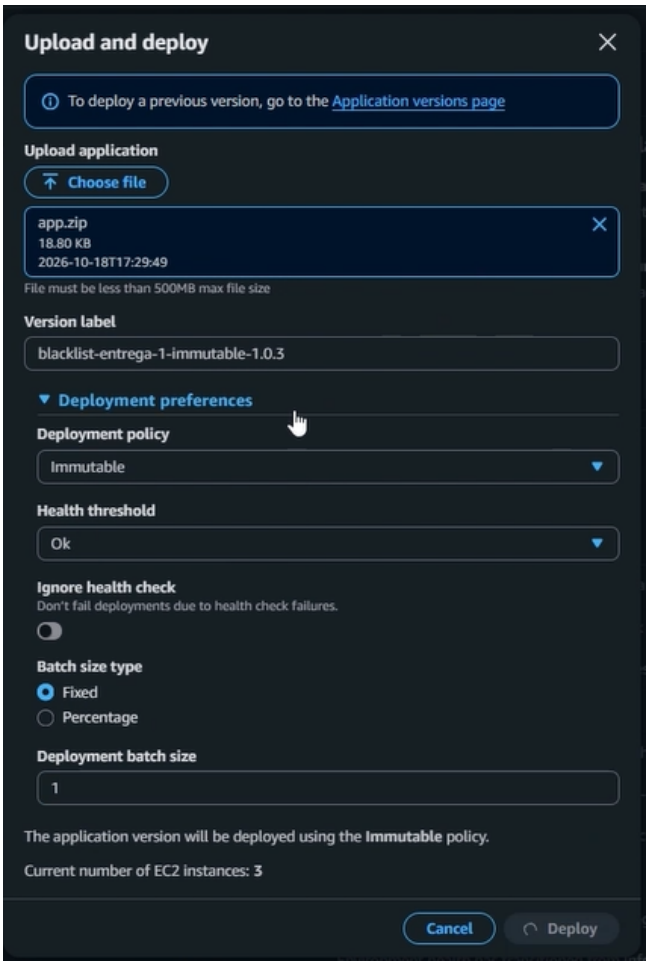
Crea un Auto Scaling group nuevo con la versión nueva; si pasa health checks, corta el viejo. Si falla, deja las instancias viejas intactas. Despliega en instancias nuevas.

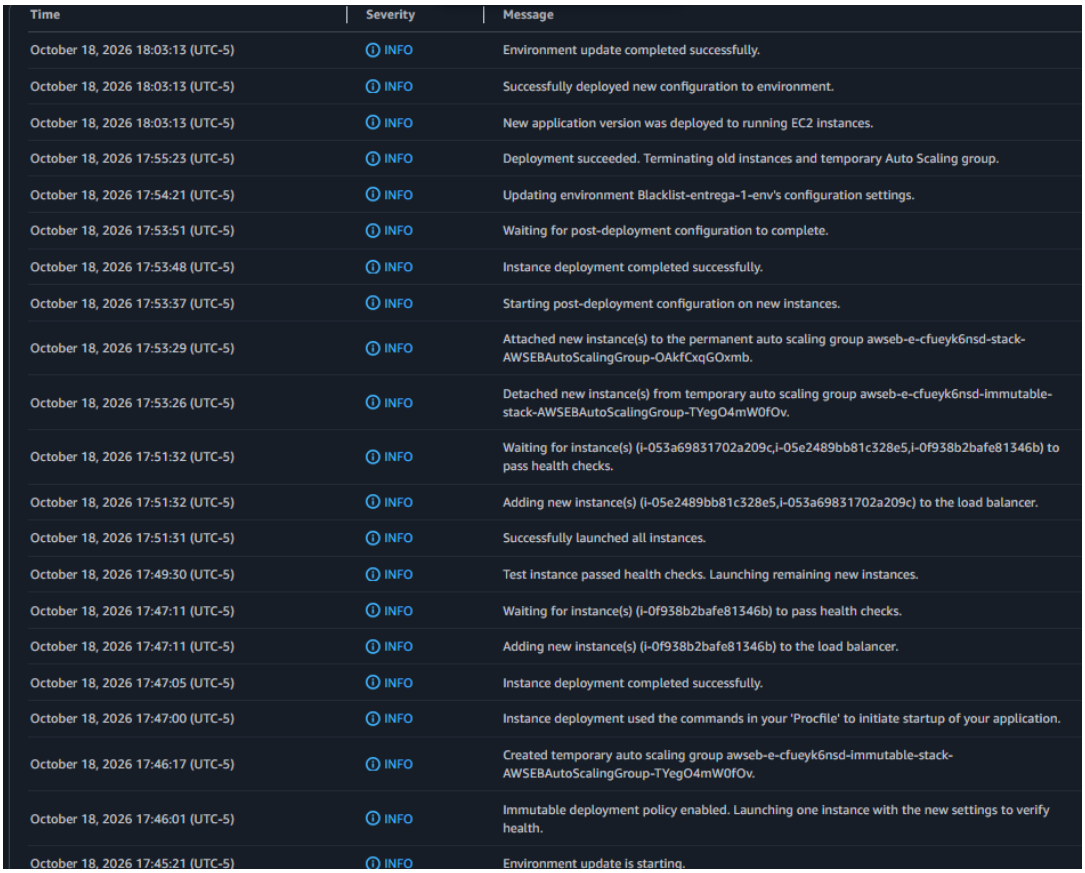
Lanza un juego completo de instancias nuevas con la versión nueva. Las viejas siguen sirviendo hasta que las nuevas pasen health check. Solo al final (si todo OK) el tráfico pasa a las nuevas y se terminan las viejas. Capacidad completa durante el deploy; si falla, las viejas quedan intactas (rollback fácil). Suele ser más lento y más caro (duplicas instancias un rato).

a) Cantidad de instancias utilizadas: mínimo 3, máximo 6.
b) Validación: con Postman se hicieron requests al endpoint base. Después del deploy: Instance IDs distintos a los iniciales y version nueva (ej. 1.0.4). Las instancias viejas fueron terminadas.


```
{
"instance": "ip-172-31-yy-yy.ec2.internal",
"service": "blacklist-microservice",
"status": "ok",
"version": "1.0.4"
}
```


c) Tiempo total de despliegue: 17m 51s
d) El despliegue se realizó en instancias nuevas.
e) Captura de pantalla del proceso de despliegue:


Upload and deploy
To deploy a previous version, go to the Application versions page
Upload application
Choose file
app.zip
18.80 KB
2026-10-18T17:29:49
File must be less than 500MB max file size
Version label
blacklist-entrega-1-immutable-1.0.3
Deployment preferences
Deployment policy
Immutable
Health threshold
Ok
Ignore health check
Don't fail deployments due to health check failures.
Batch size type
Fixed
Percentage
Deployment batch size
1
The application version will be deployed using the Immutable policy.
Current number of EC2 instances: 3
Cancel
Deploy



| Time | Severity | Message |
|---|---|---|
| October 18, 2026 18:03:13 (UTC-5) | INFO | Environment update completed successfully. |
| October 18, 2026 18:03:13 (UTC-5) | INFO | Successfully deployed new configuration to environment. |
| October 18, 2026 18:03:13 (UTC-5) | INFO | New application version was deployed to running EC2 instances. |
| October 18, 2026 17:55:23 (UTC-5) | INFO | Deployment succeeded. Terminating old instances and temporary Auto Scaling group. |
| October 18, 2026 17:54:21 (UTC-5) | INFO | Updating environment Blacklist-entrega-1-env's configuration settings. |
| October 18, 2026 17:53:51 (UTC-5) | INFO | Waiting for post-deployment configuration to complete. |
| October 18, 2026 17:53:48 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 17:53:37 (UTC-5) | INFO | Starting post-deployment configuration on new instances. |
| October 18, 2026 17:53:29 (UTC-5) | INFO | Attached new instance(s) to the permanent auto scaling group awseb-e-cfueyk6nsd-stack-AWSEBAutoScalingGroup-OAkfCxqGOxmb. |
| October 18, 2026 17:53:26 (UTC-5) | INFO | Detached new instance(s) from temporary auto scaling group awseb-e-cfueyk6nsd-immutable-stack-AWSEBAutoScalingGroup-TYegO4mW0fOv. |
| October 18, 2026 17:51:32 (UTC-5) | INFO | Waiting for instance(s) (i-053a69831702a209c,i-05e2489bb81c328e5,i-0f938b2bafe81346b) to pass health checks. |
| October 18, 2026 17:51:32 (UTC-5) | INFO | Adding new instance(s) (i-05e2489bb81c328e5,i-053a69831702a209c) to the load balancer. |
| October 18, 2026 17:51:31 (UTC-5) | INFO | Successfully launched all instances. |
| October 18, 2026 17:49:30 (UTC-5) | INFO | Test instance passed health checks. Launching remaining new instances. |
| October 18, 2026 17:47:11 (UTC-5) | INFO | Waiting for instance(s) (i-0f938b2bafe81346b) to pass health checks. |
| October 18, 2026 17:47:11 (UTC-5) | INFO | Adding new instance(s) (i-0f938b2bafe81346b) to the load balancer. |
| October 18, 2026 17:47:05 (UTC-5) | INFO | Instance deployment completed successfully. |
| October 18, 2026 17:47:00 (UTC-5) | INFO | Instance deployment used the commands in your 'Procfile' to initiate startup of your application. |
| October 18, 2026 17:46:17 (UTC-5) | INFO | Created temporary auto scaling group awseb-e-cfueyk6nsd-immutable-stack-AWSEBAutoScalingGroup-TYegO4mW0fOv. |
| October 18, 2026 17:46:01 (UTC-5) | INFO | Immutable deployment policy enabled. Launching one instance with the new settings to verify health. |
| October 18, 2026 17:45:21 (UTC-5) | INFO | Environment update is starting. |

*Ilustración 18 Proceso de despliegue Immutable*

f) Hallazgos encontrados de la estrategia: El batch size aquí controla cómo se llena la flota nueva (en lotes), no tumba la vieja por lotes como en Rolling. El más demorado de todos.

# Pruebas

El microservicio Blacklist Microservice expone una API REST desarrollada en Python con Flask y Flask-RESTful, que permite registrar y consultar direcciones de correo electrónico en la lista negra global de la organización. La información se almacena en PostgreSQL y el acceso a los endpoints está protegido mediante autenticación con JWT.

La API dispone de dos endpoints principales:

- **POST /blacklists**: registra un correo electrónico en la lista negra.
- **GET /blacklists/<email>**: consulta si un correo electrónico se encuentra en la lista negra y obtiene el motivo asociado, cuando existe.

Todas las solicitudes deben incluir el encabezado de autorización Authorization: Bearer TOKEN, utilizando un token JWT válido.

Respuestas:

- **201 Created**: Correo registrado correctamente en la lista de bloqueo.
- **200 OK**: Consulta realizada correctamente.
- **400 Bad Request**: El campo blocked_reason supera los 255 caracteres permitidos.
- **401 Unauthorized**: Token JWT ausente o inválido.
- **404 Not Found**: El correo consultado no existe, si así lo define la API.
- **422 Unprocessable Entity**: Los datos enviados no cumplen las reglas de validación, si así está configurada la API.
- **500 Internal Server Error**: Error inesperado en el servidor.
- **Signature verification failed**: El JWT tiene una firma inválida o fue generado con una clave secreta diferente.
- **Not enough segments**: El JWT está incompleto o mal formado; debe contener tres segmentos separados por puntos.

Variables:

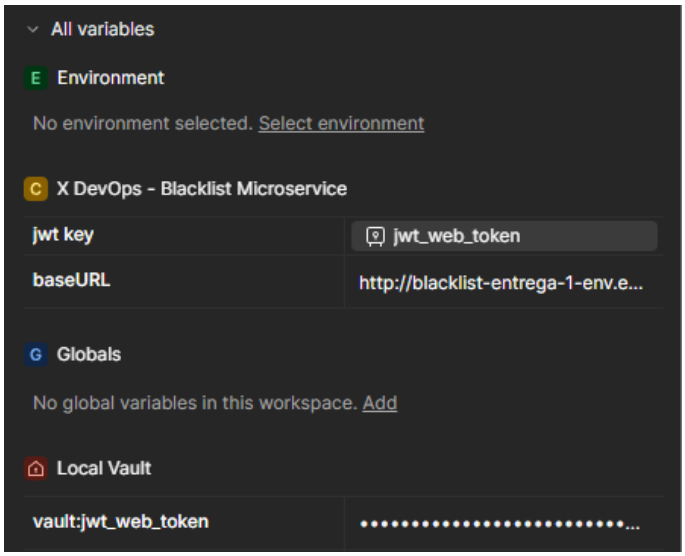



*Ilustración 19 Variables*

Para comprobar el funcionamiento de la API se prepararon solicitudes en Postman que permiten validar los casos exitosos y las situaciones de error:

- Registrar un correo duplicado:

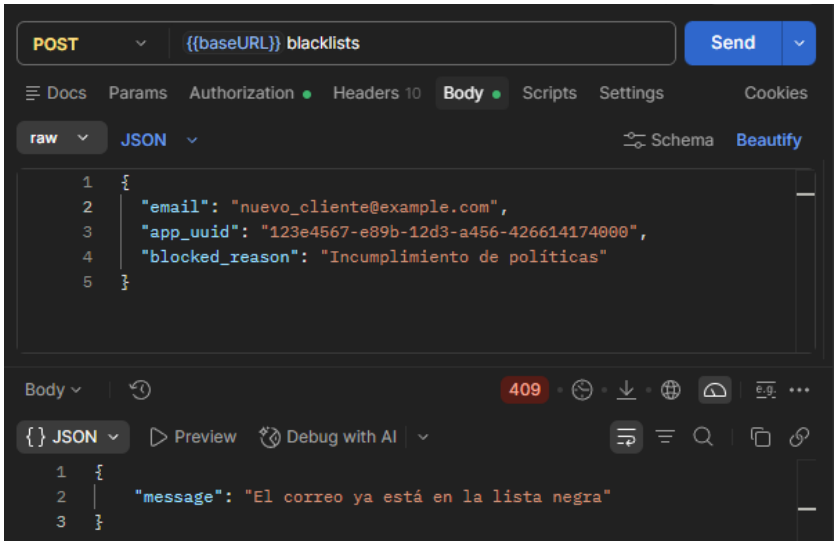



*Ilustración 20 Resultado registrar un correo duplicado*

- Registrar correo:

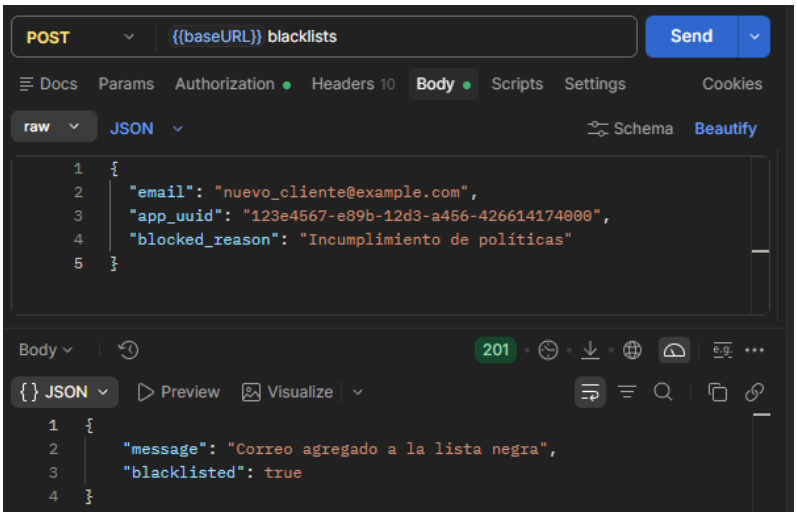



*Ilustración 21 Resultado registrar un correo*

- Consultar correo bloqueado:

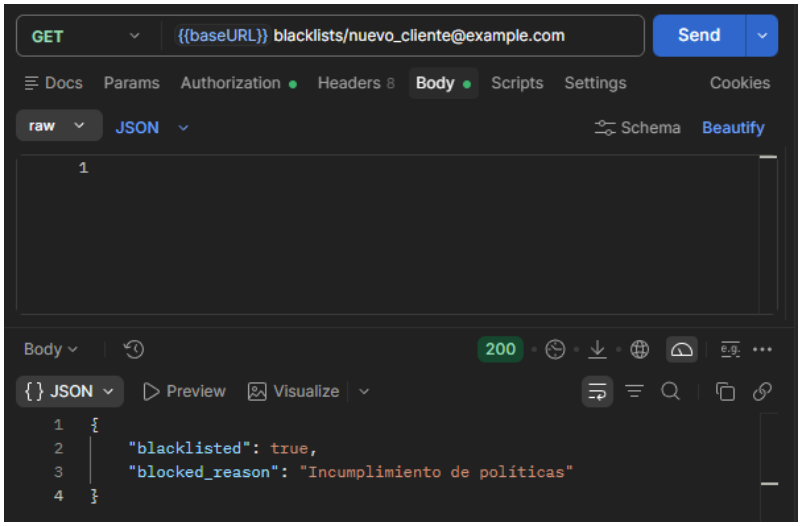



*Ilustración 22 Resultado consultar correo bloqueado*

- Consultar un correo inexistente:

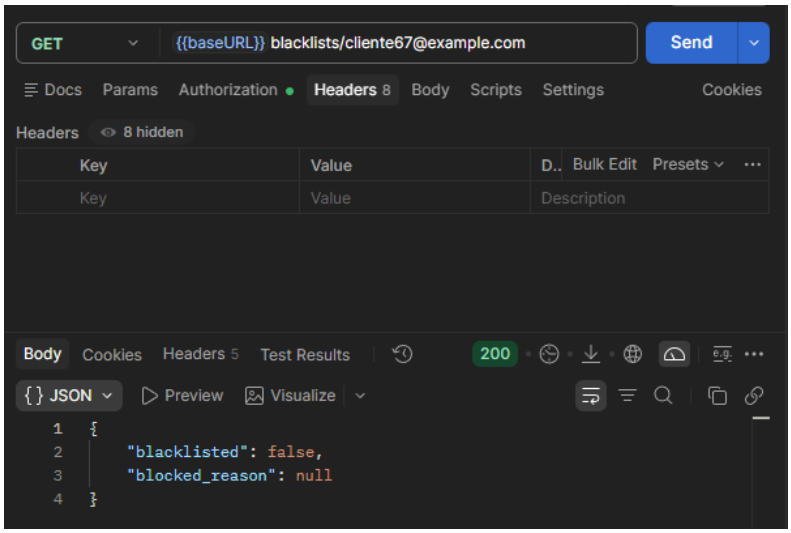



*Ilustración 23 Resultado consultar un correo inexistente*

- Enviar un UUID inválido:

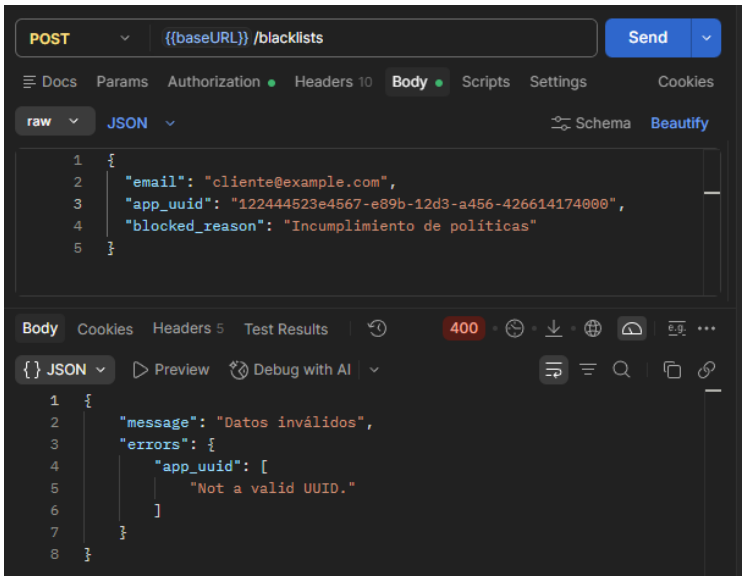
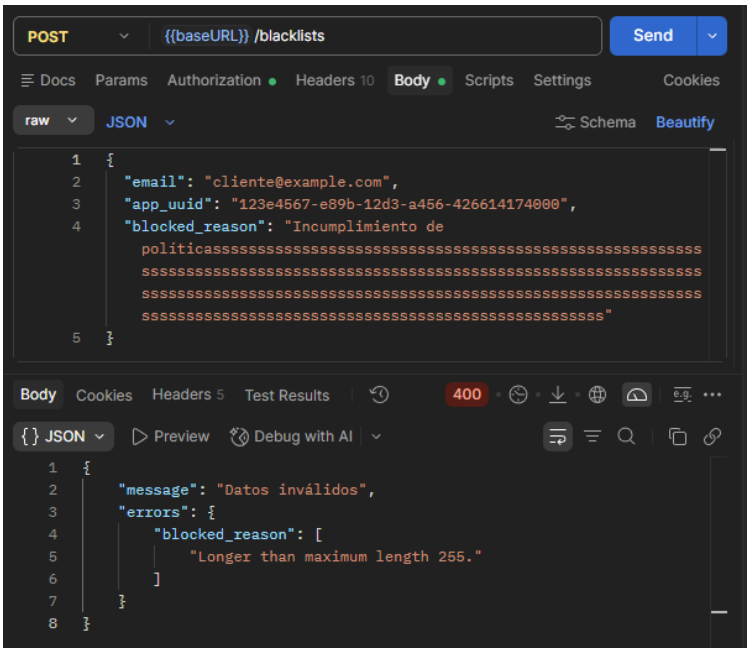



*Ilustración 24 Resultado enviar un UUID inválido*

- Enviar un motivo de más de 255 caracteres:


POST
{{baseURL}} /blacklists
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"email": "cliente@example.com",
"app_uuid": "123e4567-e89b-12d3-a456-426614174000",
"blocked_reason": "Incumplimiento de
politicasssssss

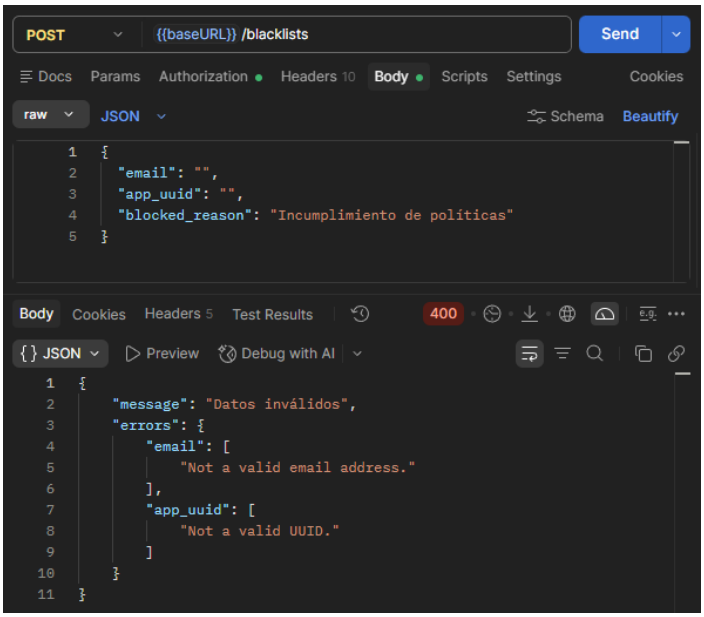



*Ilustración 26 Resultado omitir email o app_uuid*

# Enlaces

- URL Documentación Postman: https://documenter.getpostman.com/view/30377337/2sBYHSx12K
- URL Git Proyecto X DevOps: https://github.com/cchantryt/blacklist-microservice
- Video sustentación: https://uniandes-my.sharepoint.com/:v:/g/personal/s_castrod_uniandes_edu_co/IQBUlhi1CdxdS6ezqn2DHIo6AYALy8wQcrZCe_9ltRaJQeA?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=x6o6z7